

TDT4195 Exercise 2

Kacper Krzysztof Maciejko

Clément Jourdin

September 17, 2026

1 Task 1: Per-Vertex Colors

1.1 (a)

Most of this task we have already accidentally accomplished in Assignment 1. What needed to be changed was adding the alpha value, instead of setting it to 1.0 in the fragment shader as done previously.

1.1.0.1 Modifying `create_vao` Function creating VAO needed to be modified to accept each vertice containing 7 floating point values (3 describing position, 4 describing colour). To do that `gl::VertexAttribPointer` needed to be modified to be of size of 28 bytes instead of the previous 12. Additionally, it needed to be called twice, once for the position values, and once with colour values, with the last passed value being an offset of 3 bytes (as described in Assignment 1)

1.1.0.2 Modifying shaders Vertex shaders needed to be modified to pass a vector containing color to the fragment shader. Fragment shader needed to be modified to take that as input instead of colour being set once and for all in its code.



1.1.0.3 Colours between vertices Colours between vertices are an interpolation of colours assigned to each vertex. This creates a smooth gradient, like one seen in the image above.

1.2 Task 2: Alpha Blending and Depth

1.3 (a)

```
// Triangles for Assignment 2 Task 2
let vertices3_vec_4: Vec<f32> = vec![
    -0.6, -0.6, 0.9, 1.0, 0.0, 0.0, 0.5,
    0.6, -0.6, 0.9, 1.0, 0.0, 0.0, 0.5,
    0.0, 0.6, 0.9, 1.0, 0.0, 0.0, 0.5,

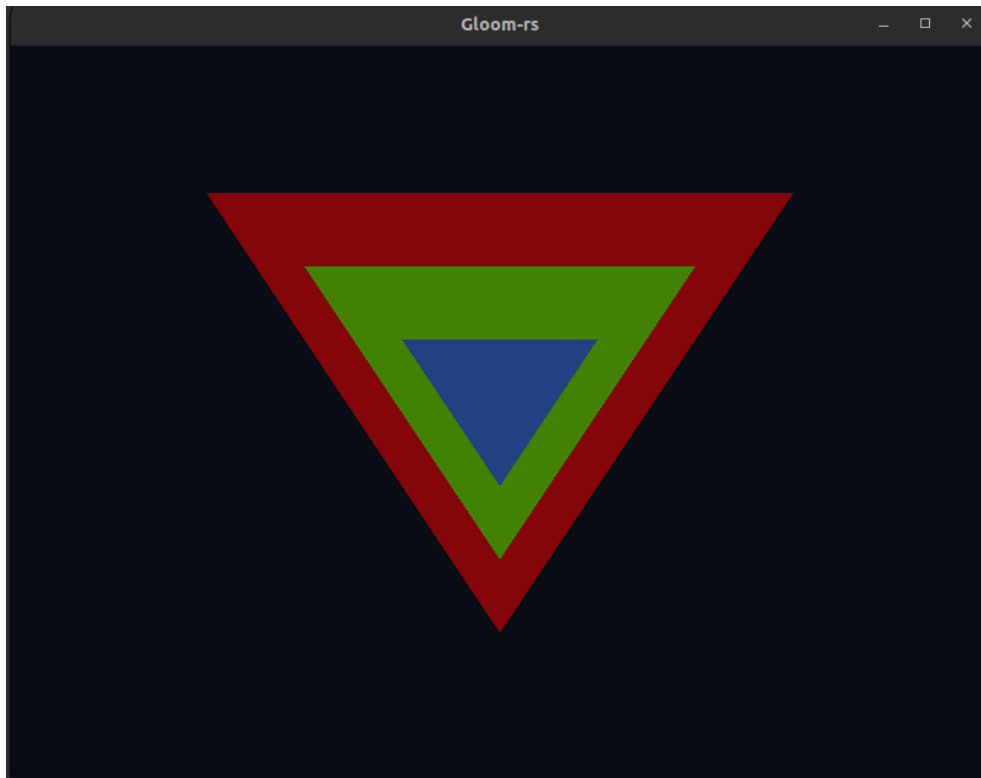
    -0.4, -0.4, 0.5, 0.0, 1.0, 0.0, 0.5,
    0.4, -0.4, 0.5, 0.0, 1.0, 0.0, 0.5,
    0.0, 0.4, 0.5, 0.0, 1.0, 0.0, 0.5,

    -0.2, -0.2, 0.0, 0.0, 0.0, 1.0, 0.5,
    0.2, -0.2, 0.0, 0.0, 0.0, 1.0, 0.5,
    0.0, 0.2, 0.0, 0.0, 0.0, 1.0, 0.5,
];

// adding more triangles, we also have to add more indices (3 for each)
let indices3_vec_4: Vec<u32> = vec![
    0, 1, 2,
    3, 4, 5,
```

```
6, 7, 8,
];
```

```
let my_vao3 = unsafe { create_vao(&vertices3_vec_4, &indices3_vec_4) };
```



1.4 (b)

1.4.1 (i)

// Triangles for Assignment 2 Task 2

```
let vertices3_vec_4: Vec<f32> = vec![
    -0.6, -0.6, 0.9, 0.0, 0.0, 1.0, 0.5,
    0.6, -0.6, 0.9, 0.0, 0.0, 1.0, 0.5,
    0.0, 0.6, 0.9, 0.0, 0.0, 1.0, 0.5,

    -0.4, -0.4, 0.5, 1.0, 0.0, 0.0, 0.5,
    0.4, -0.4, 0.5, 1.0, 0.0, 0.0, 0.5,
    0.0, 0.4, 0.5, 1.0, 0.0, 0.0, 0.5,

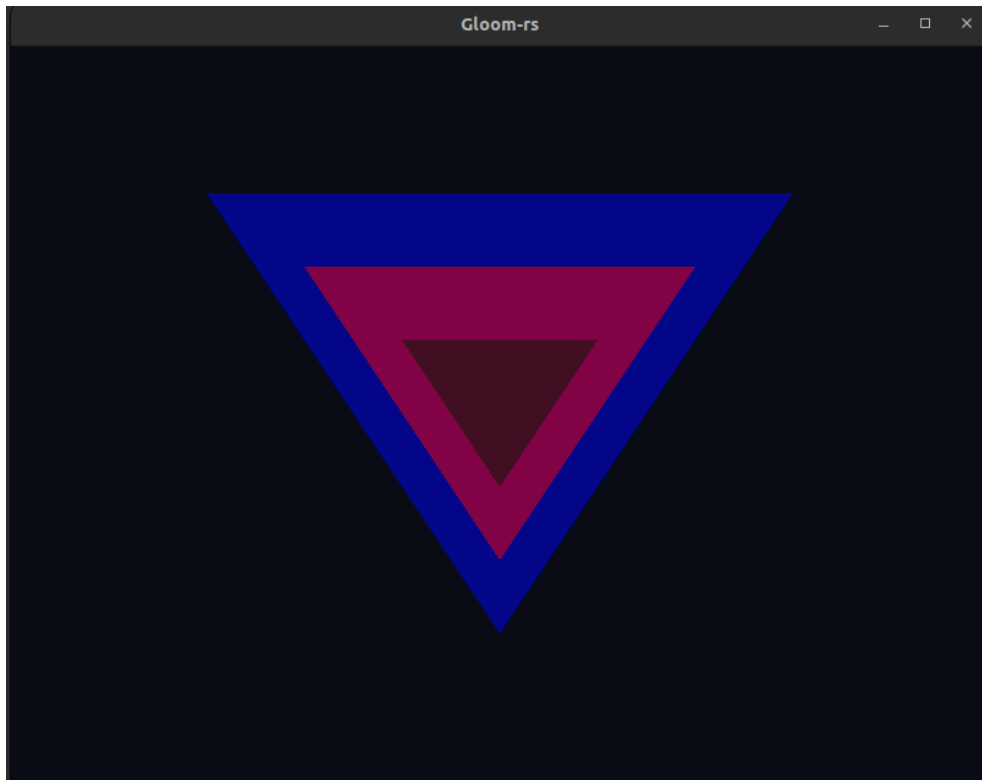
    -0.2, -0.2, 0.0, 0.0, 0.1, 0.0, 0.5,
    0.2, -0.2, 0.0, 0.0, 0.1, 0.0, 0.5,
    0.0, 0.2, 0.0, 0.0, 0.1, 0.0, 0.5,
];
```

// adding more triangles, we also have to add more indices (3 for each)

```
let indices3_vec_4: Vec<u32> = vec![
```

```
0, 1, 2,
3, 4, 5,
6, 7, 8,
];
```

```
let my_vao3 = unsafe { create_vao(&vertices3_vec_4, &indices3_vec_4) };
```



When swapping the colors of the three triangles, the color of the area in which the triangles overlap changes as well. This can be explained by the following equation:

$$\text{Color_New} = \text{Color_Source} * \text{Alpha_Source} + \text{Color_Destination} * (1 - \text{Alpha_Source})$$

Let's apply this equation to a pixel in the overlapping area in both cases and witness the difference. In the first case (question (a)), we have: - When the first (red) triangle is drawn: $\text{Color_New} = 0.5 * \text{red} + 1 * \text{black}$ - When the second (green) triangle is drawn: $\text{Color_New} = 0.5 * \text{green} + 0.5 * (0.5 * \text{red} + 1 * \text{black}) = 0.5 * \text{green} + 0.25 * \text{red} + 0.5 * \text{black}$ - When the last (blue) triangle is drawn: $\text{Color_New} = 0.5 * \text{blue} + 0.5 * (0.5 * \text{green} + 0.25 * \text{red} + 0.5 * \text{black}) = 0.5 * \text{blue} + 0.25 * \text{green} + 0.125 * \text{red} + 0.25 * \text{black}$

Whereas in the second case (question (b)), we have: - When the first (blue) triangle is drawn: $\text{Color_New} = 0.5 * \text{blue} + 1 * \text{black}$ - When the second (red) triangle is drawn: $\text{Color_New} = 0.5 * \text{red} + 0.5 * (0.5 * \text{blue} + 1 * \text{black}) = 0.5 * \text{red} + 0.25 * \text{blue} + 0.5 * \text{black}$ - When the last (green) triangle is drawn: $\text{Color_New} = 0.5 * \text{green} + 0.5 * (0.5 * \text{red} + 0.25 * \text{blue} + 0.5 * \text{black}) = 0.5 * \text{green} + 0.25 * \text{red} + 0.125 * \text{blue} + 0.25 * \text{black}$

$0.5 * \text{blue} + 0.25 * \text{green} + 0.125 * \text{red} + 0.25 * \text{black} \neq 0.5 * \text{green} + 0.25 * \text{red} + 0.125 * \text{blue} + 0.25 * \text{black}$, therefore the overlapping area appears in a different color in each case.

1.4.2 (ii)

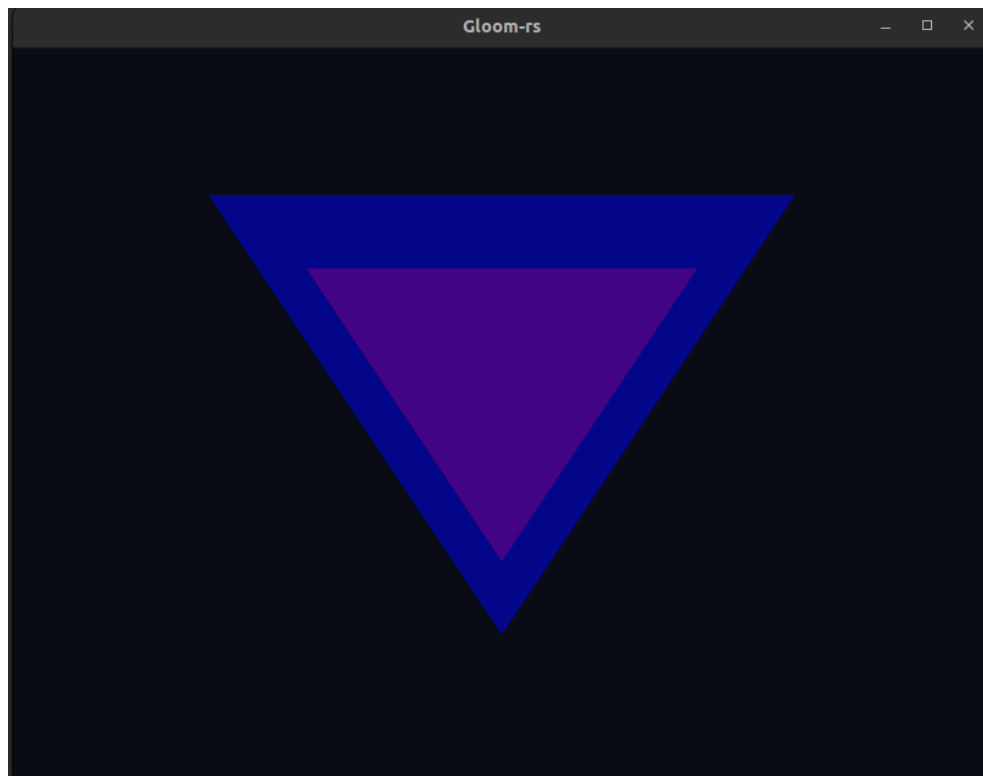
```
// Triangles for Assignment 2 Task 2
let vertices3_vec_4: Vec<f32> = vec![
    -0.6, -0.6, 0.0, 0.0, 0.0, 1.0, 0.5,
    0.6, -0.6, 0.0, 0.0, 0.0, 1.0, 0.5,
    0.0, 0.6, 0.0, 0.0, 0.0, 1.0, 0.5,

    -0.4, -0.4, 0.5, 1.0, 0.0, 0.0, 0.5,
    0.4, -0.4, 0.5, 1.0, 0.0, 0.0, 0.5,
    0.0, 0.4, 0.5, 1.0, 0.0, 0.0, 0.5,

    -0.2, -0.2, 0.9, 0.0, 0.1, 0.0, 0.5,
    0.2, -0.2, 0.9, 0.0, 0.1, 0.0, 0.5,
    0.0, 0.2, 0.9, 0.0, 0.1, 0.0, 0.5,
];

// adding more triangles, we also have to add more indices (3 for each)
let indices3_vec_4: Vec<u32> = vec![
    6, 7, 8,
    3, 4, 5,
    0, 1, 2,
];

let my_vao3 = unsafe { create_vao(&vertices3_vec_4, &indices3_vec_4) };
```



When swapping the z-coordinate of the three triangles, the color of the area in which the triangles overlap changes. We notice that the furthest a triangle is, the less its color impacts the color of the overlapping area. In the example given in the screenshot, the triangle with the biggest z-coordinate (ie the furthest one) is the green one, and green is also the less prominent color in the overlapping area. Those observations make sense as we can expect to see the objects that are close to the “camera” better than the ones that are far from it.

2 Task 3: The Affine Transformation Matrix

2.0.1 (a)

Completing this task required modifying the Vertex Shader in a manner shown below:

```
#version 430 core

layout (location = 0) in vec3 position;
layout (location = 1) in vec4 color;

out VS_OUTPUT {
    vec4 color;
} OUT;

void main()
{
    mat4x4 matrix = {
        {1.0, 0.0, 0.0, 0.0},
        {0.0, 1.0, 0.0, 0.0},
        {0.0, 0.0, 1.0, 0.0},
        {0.0, 0.0, 0.0, 1.0}
    };
    gl_Position = matrix * vec4(position, 1.0f);
    OUT.color = color;
}
```

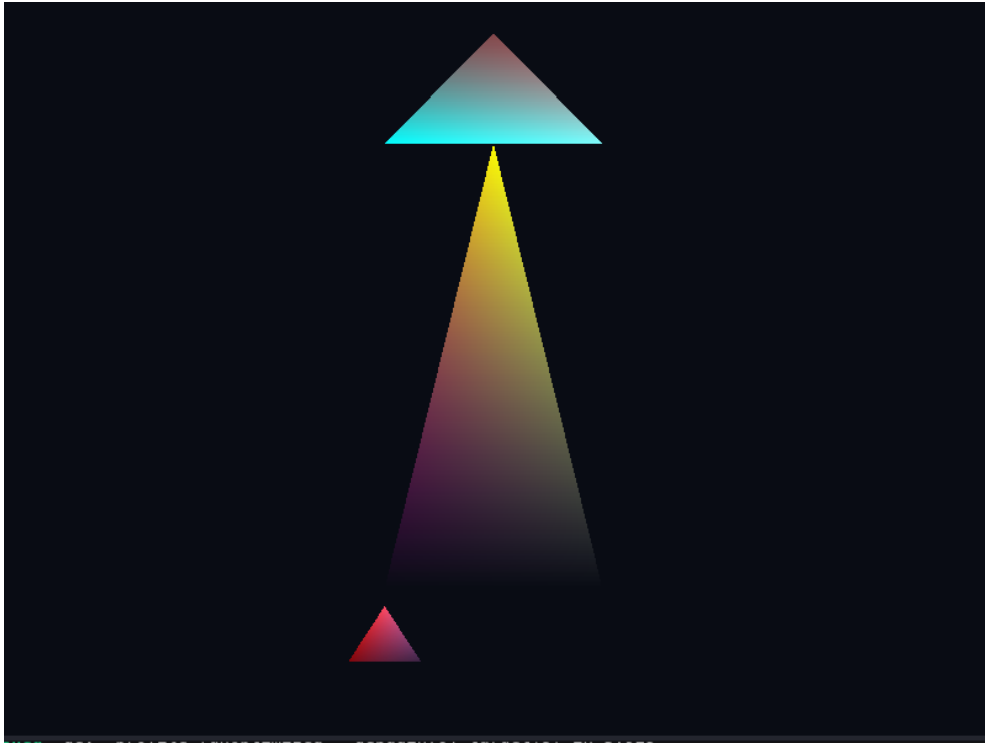
The vertices multiplied by the 4x4 identity matrix result in no changes being made to the original result:



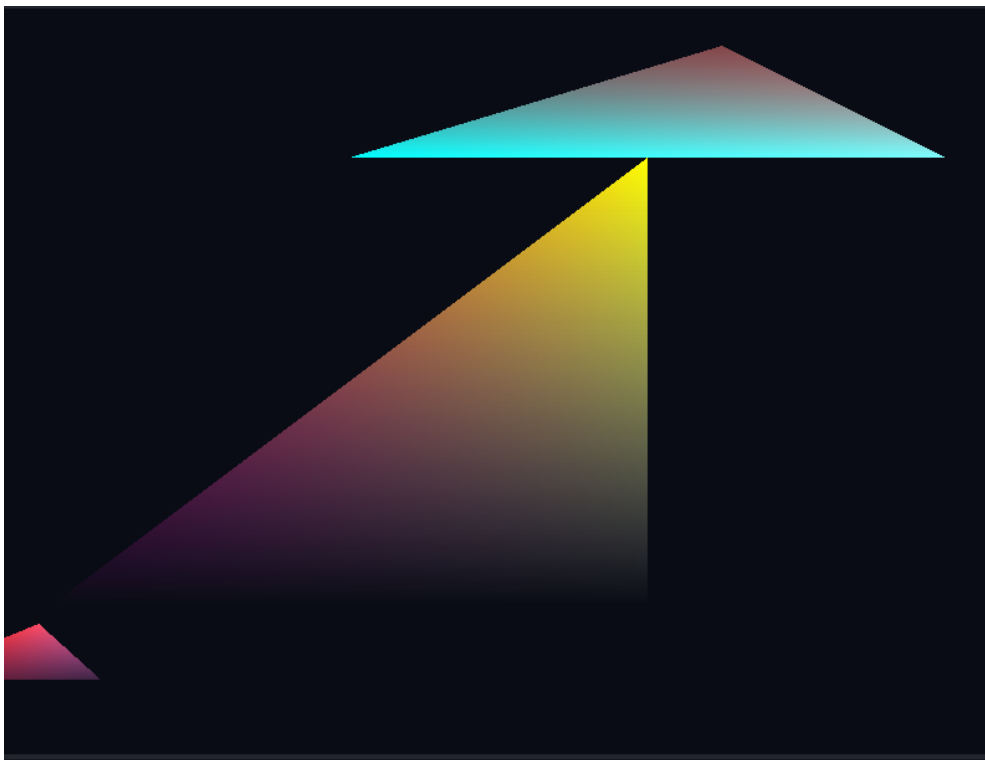
2.0.2 (b)

Modifying each value marked with letters in the matrix:

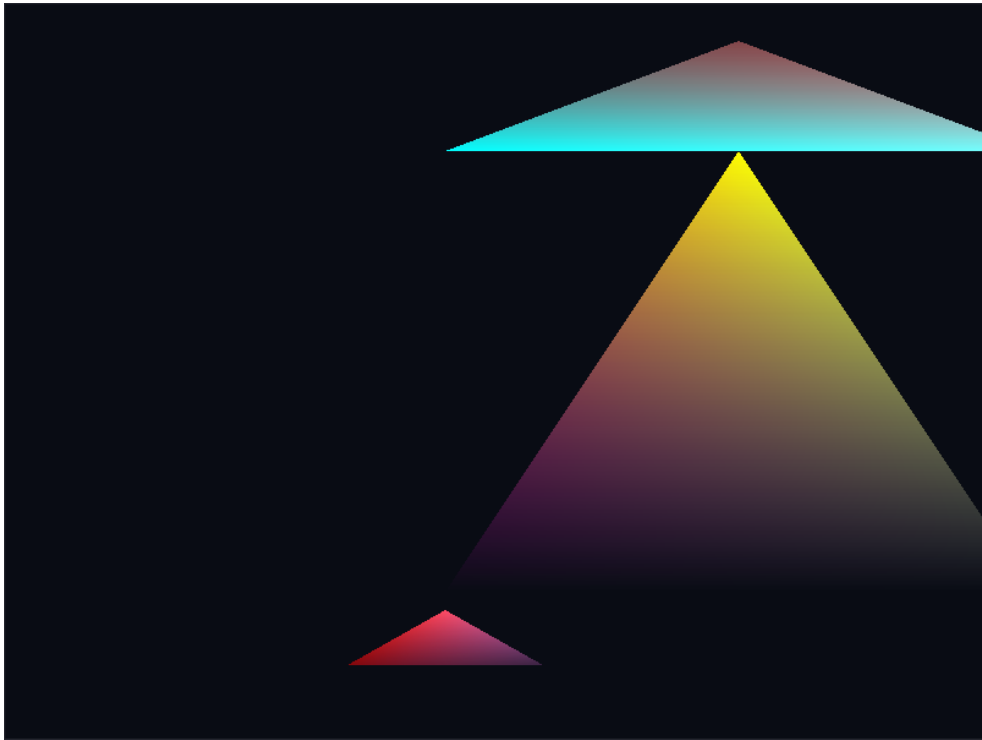
2.0.2.1 a Modifying this value results in scaling along the x axis:



2.0.2.2 b Modifying this value results in shearing along the x axis:



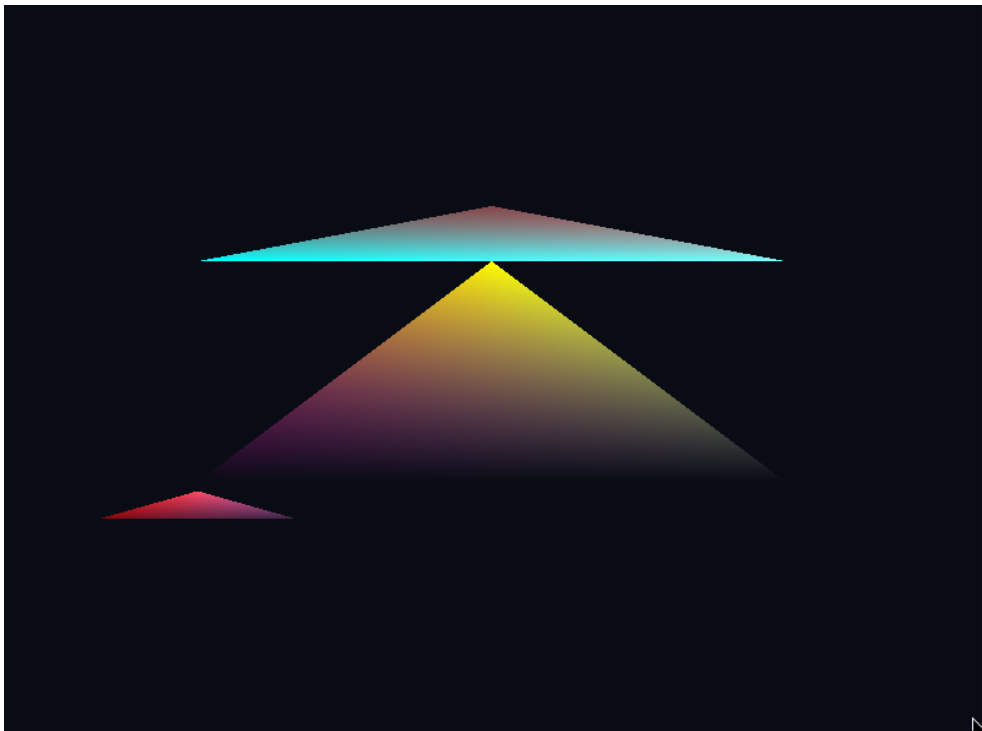
2.0.2.3 c Modifying this value results in translation along the x axis:



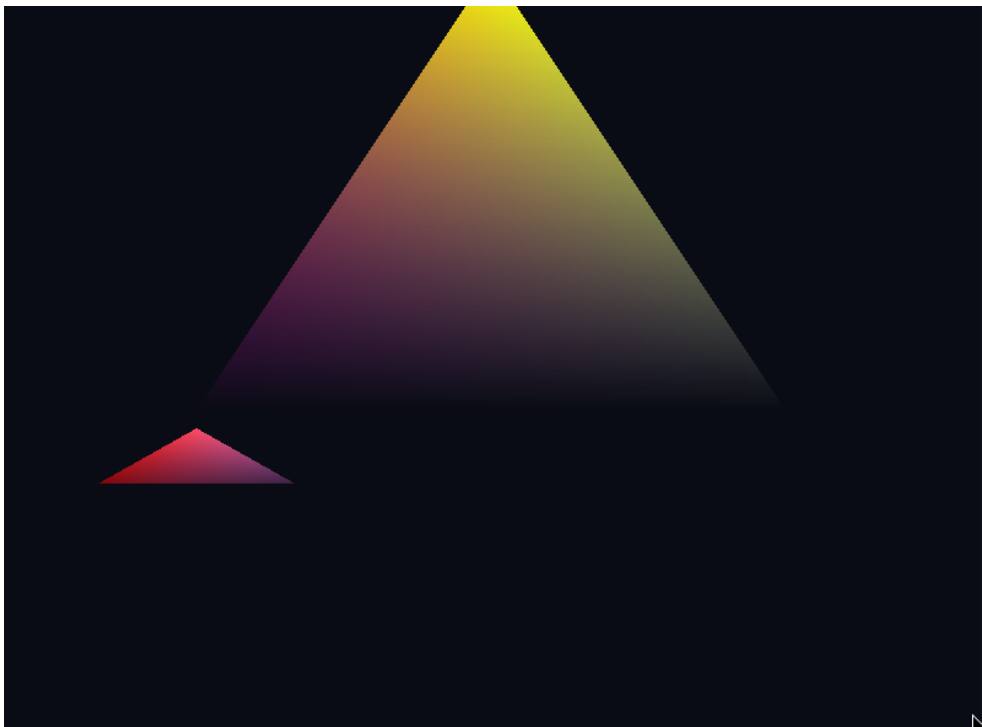
2.0.2.4 d Modifying this value results in shearing along the y axis:



2.0.2.5 e Modifying this value results in scaling along the y axis:



2.0.2.6 f Modifying this value results in translation along the y axis:

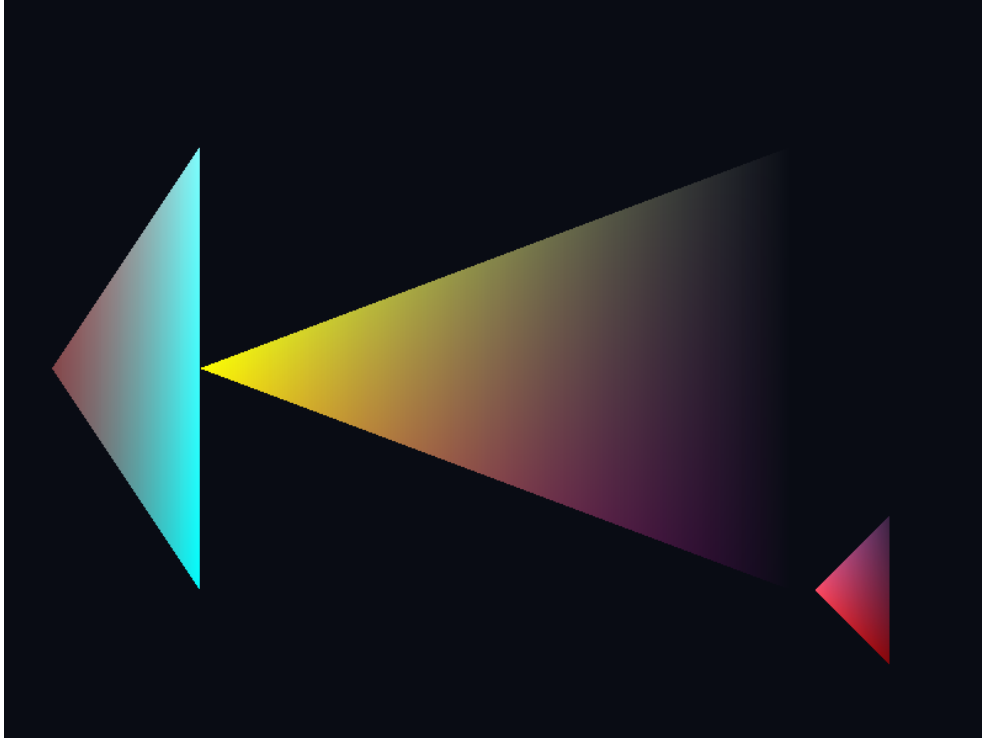


2.0.3 (c)

Why can you be certain that none of the observed transformations were rotations?

None of the observed transformations were rotations, because:

- * The distance from the origin does not change during a rotation. In all observed transformations the distance changed.
- * The rotation matrix is defined by: $\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$ We know that none of the transformations are rotations, because at least two values would have to be changed to achieve that.



3 Task 4: Combinations of Transformations

3.1 (a) Passing the transformation matrix as a uniform variable

3.1.1 main.rs

```
unsafe {  
    let loc = simple_shader.get_uniform_location("camera_transformation_matrix");  
    gl::UniformMatrix4fv(loc, 1, gl::FALSE, camera_transformation_matrix.as_ptr());  
}
```

3.1.2 Vertex Shader

```
#version 430 core  
  
layout (location = 0) in vec4 position;  
layout (location = 1) in vec4 color;
```

```

uniform mat4x4 camera_transformation_matrix;

out VS_OUTPUT {
    vec4 color;
} OUT;

void main()
{
    vec4 newPosition = camera_transformation_matrix*position;
    gl_Position = newPosition;
    OUT.color = color;
}

```

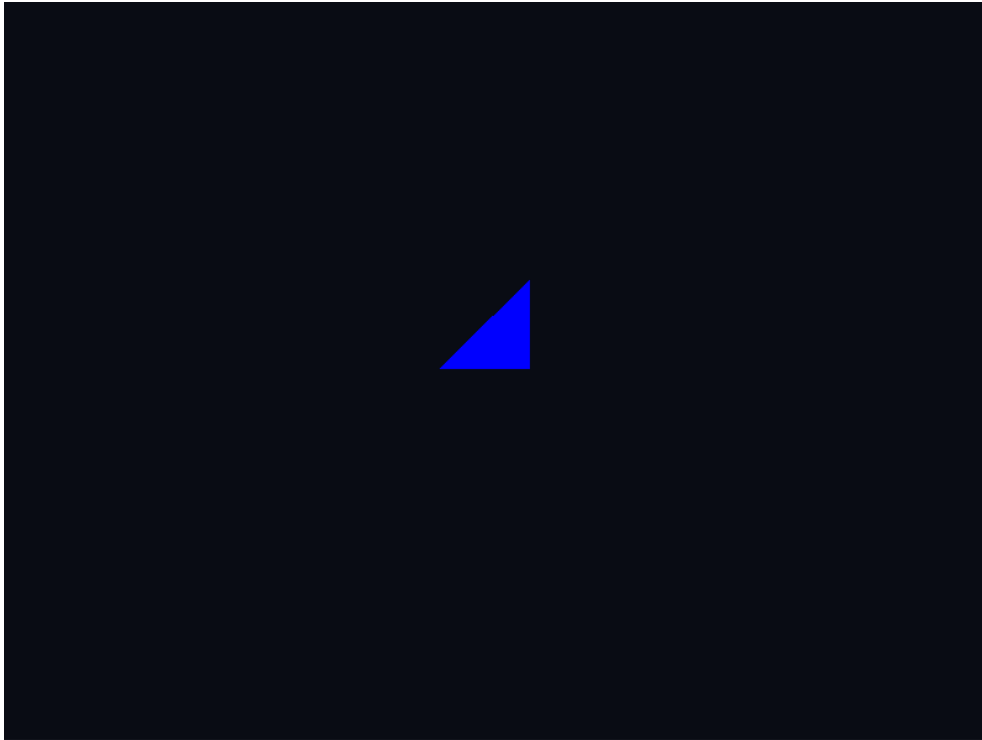
3.2 (b) Applying projection

```

let camera_projection_matrix: glm::Mat4 =
    glm::perspective(
        window_aspect_ratio,
        (std::f32::consts::PI)/2.0,
        1.0,
        100.0,
    );

// The final transformation matrix is a combination of all the previously set
// transformation matrix
let camera_transformation_matrix =
    camera_projection_matrix
    * camera_rotation_matrix_y
    * camera_rotation_matrix_x
    * camera_translation_matrix;

```



4 (c) Creating a camera

For this task, we have applied a transformation matrix, moving the world around the camera in order to imitate the camera movement.

Movement is stored in variables defined below:

```
let mut cameraX = 0.0;
let mut cameraY = 0.0;
let mut cameraZ = 0.0;
let mut angleX = 0.0;
let mut angleY = 0.0;
```

Key handler has also been added for controlling the motion:

```
if let Ok(keys) = pressed_keys.lock() {
    for key in keys.iter() {
        match key {
            VirtualKeyCode::W => {
                cameraZ += delta_time*camera_speed;
            }
            VirtualKeyCode::S => {
                cameraZ -= delta_time*camera_speed;
            }
            VirtualKeyCode::A => {
                cameraX += delta_time*camera_speed;
            }
            VirtualKeyCode::D => {
```

```

        cameraX -= delta_time*camera_speed;
    }
    VirtualKeyCode::Space => {
        cameraY += delta_time*camera_speed;
    }
    VirtualKeyCode::LShift => {
        cameraX -= delta_time*camera_speed;
    }
    VirtualKeyCode::Left => {
        angleY += delta_time*camera_speed;
    }
    VirtualKeyCode::Up => {
        angleX += delta_time*camera_speed;
    }
    VirtualKeyCode::Right => {
        angleY -= delta_time*camera_speed;
    }
    VirtualKeyCode::Down => {
        angleX -= delta_time*camera_speed;
    }

    // default handler:
    _ => {}
}
}
}

```

The movement of the camera is controlled by WSAD, space, LShift and arrow keys, as recommended in instruction.

4.0.1 Generating a transformation matrix

```

let mut camera_transformation_matrix : glm::Mat4 = glm::identity();
let camera_translation_matrix: glm::Mat4 =
    glm::translation(&glm::vec3(cameraX, cameraY, cameraZ));
let camera_rotation_matrix_x: glm::Mat4 =
    glm::rotation(-angleX, &glm::vec3(1.0, 0.0, 0.0));

let camera_rotation_matrix_y: glm::Mat4 =
    glm::rotation(-angleY, &glm::vec3(0.0, 1.0, 0.0));

let camera_projection_matrix: glm::Mat4 =
    glm::perspective(
        window_aspect_ratio,
        (std::f32::consts::PI)/2.0,
        1.0,
        100.0,
    );

// The final transformation matrix is a combination of all the previously set
// transformation matrix
let camera_transformation_matrix =

```

```
camera_projection_matrix  
* camera_rotation_matrix_y  
* camera_rotation_matrix_x  
* camera_translation_matrix;
```